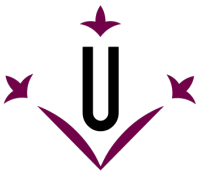

**Universitat de Lleida**

Degree in Computer Engineering

Subject: Web Project

# Deliverable 2

## Choosy Web 2.0 Application

Academic Year 2025/26

Group: choosy-friends-app

**Team members**
Aleix Bertran
Bru Pallàs
Amaru Alviña
Aleix Rosinach

May 2026

# Contents

# Deliverable 2: Web 2.0 Evolution of Choosy

This document describes the changes made to the Choosy application for the second delivery of the Web Project subject. The first delivery defined the domain model, application architecture, authentication, Docker deployment, and the base group decision-making workflow. This second delivery extends that application with Web 2.0 features for end users: authenticated creation, edition, deletion, end-to-end validation, security restrictions, and external API data integrated into the plan edition experience.

The implementation is available in the same GitHub repository used for Deliverable 1.

## 1. GitHub Repository

- Public repository URL: https://github.com/choosy-friends-app/choosy-app
- Working branch for this delivery: `second_activity`
- The `db.sqlite3` file is committed at the repository root to simplify application testing and evaluation.
- Access has been prepared for the GitHub user requested in the statement: `rogargon`.

## 2. Users for Evaluation

### 2.1 Admin User

- Username: `admin`
- Password: `admin1234`

### 2.2 Functional Demo Users

- `aleix` / `demo1234`
- `bru` / `demo1234`
- `eldejuneda` / `demo1234`
- `chileno` / `demo1234`

These users allow the evaluator to check normal user flows without using the Django administration interface.

### 2.3 Automatic User Provisioning

When the application is started with `docker-compose up` or `podman-compose up`, the `web` service executes:

1. `python manage.py migrate`
2. `python manage.py seed_demo_users`
3. `gunicorn ...`

This guarantees that migrations and demo users are available in a fresh environment.

## 3. Summary of Changes Since Deliverable 1

Deliverable 1 implemented the initial Choosy concept: groups, members, proposals, plan options, votes, notifications, authentication, Docker support, and a modular Django structure. Deliverable 2 keeps that architecture and adds the following user-facing features:

- CRUD operations for the main entities that make sense for registered users to manage: `Group`, `PlanProposal`, and `Plan`.
- Ownership-based security so users can only modify or delete their own entities.
- ModelForm-based validation in public application views, without relying on Django admin.
- Class-Based Views for create, update, and delete workflows.
- E2E tests with Behave and Splinter covering success cases, validation errors, and unauthorized access.

- External API data from OpenStreetMap Nominatim and Open-Meteo integrated into the plan form through asynchronous browser requests.
- Improved plan creation UX with location autocomplete and weather preview before saving an option.

No fundamental domain replacement was made. The second delivery builds directly on the first-delivery model and adds the required Web 2.0 layer around it.

# 4. Web 2.0 CRUD Features

## 4.1 Entity Creation

Registered users can create the following entities directly from the application:

- `Group`: a user can create a new group, automatically becoming its owner and an active member.
- `PlanProposal`: a user can create a new proposal for one of their owned groups.
- `Plan`: a user can add a plan option to one of their open proposals.

The implementation uses Django `CreateView` classes and ModelForms:

- `GroupCreateView`
- `PlanProposalCreateView`
- `PlanCreateView`
- `GroupForm`
- `PlanProposalForm`
- `PlanForm`

Relevant routes:

- `/crear-grupo/`
- `/crear-propuesta/`
- `/crear-plan/`

## 4.2 Entity Modification

Registered users can update only the entities they are allowed to control:

- A group can be edited only by its owner.
- A proposal can be edited only by the user that created it.
- A plan option can be edited only by the user that created it, and only while the proposal is still open.

The implementation uses Django `UpdateView` classes:

- `GroupUpdateView`
- `PlanProposalUpdateView`
- `PlanUpdateView`

Relevant routes:

- `/editar-grupo/<id>/`
- `/editar-propuesta/<id>/`
- `/editar-plan/<id>/`

## 4.3 Entity Deletion

Users can delete the same kind of instances they can modify:

- Own groups.
- Own proposals.
- Own plan options, as long as the proposal is still open.

The implementation uses Django `DeleteView` classes:

- `GroupDeleteView`

- `PlanProposalDeleteView`
- `PlanDeleteView`

Relevant routes:

- `/eliminar-grupo/<id>/`
- `/eliminar-propuesta/<id>/`
- `/eliminar-plan/<id>/`

# 5. Security and Integrity Decisions

The CRUD implementation includes explicit security restrictions because the application contains shared group data. The most important decisions are:

- All CRUD views require authentication through `LoginRequiredMixin`.
- Update and delete operations use `OwnerRequiredMixin`, returning HTTP 403 when a user tries to modify another user's entity.
- `GroupUpdateView` and `GroupDeleteView` check the `owner` field.
- `PlanProposalUpdateView`, `PlanProposalDeleteView`, `PlanUpdateView`, and `PlanDeleteView` check the `created_by` field.
- Proposal choices in forms are filtered by user, preventing users from creating plans inside proposals that belong to someone else.
- Closed proposals cannot receive new plan options.
- Existing plan options in closed proposals cannot be edited or deleted.
- Plan creation handles possible option-order conflicts and returns a form error instead of causing a server error.

These restrictions are important because Choosy is collaborative: one user should be able to participate in group decisions without being able to overwrite another user's proposals or plan options.

# 6. External API Integration

Deliverable 2 incorporates external data into the application using AJAX-assisted forms. This is implemented in the plan creation and edition form.

## 6.1 APIs Used

Two external APIs are used:

- OpenStreetMap Nominatim: location search and address normalization.
- Open-Meteo: daily weather forecast for the selected place and date.

Backend proxy endpoints:

- `GET /api/locations/search/`
- `GET /api/weather/forecast/`

The endpoints are authenticated and return normalized JSON responses to the browser. This avoids exposing application logic directly in the frontend and allows backend-side validation and error handling.

## 6.2 User Experience

When a user creates or edits a plan:

1. The user types a city, venue, or address in the location field.
2. The browser requests matching locations from the backend endpoint.
3. The user selects one result.
4. The form stores the selected latitude and longitude in hidden fields.
5. If the user selects a date, the application requests a weather forecast.
6. The forecast is shown as a preview to help the user decide whether the plan is appropriate.

This feature improves the editing process with external data instead of using the API as a decorative element.

### 6.3 Error Handling

The integration handles relevant error cases:

- Queries shorter than three characters return an empty location result list.
- Invalid coordinates return a `400` response.
- Unsupported forecast dates return a controlled `422` response.
- External API unavailability returns a controlled `502` response.
- Malformed or incomplete API data is ignored instead of breaking the form.

## 7. End-to-End Tests

End-to-end tests were added using Behave and Splinter, following the recommendation from the subject statement and the `myrestaurants` example project.

Files:

- `features/crud.feature`
- `features/steps/crud_steps.py`
- `features/environment.py`

The E2E tests cover:

- Successful creation, update, and deletion of `Plan`.
- Successful creation, update, and deletion of `Group`.
- Successful creation, update, and deletion of `PlanProposal`.
- Required-field validation errors.
- Invalid numeric data in the plan form.
- Forbidden edition of another user's plan, group, or proposal.
- Forbidden deletion of another user's plan, group, or proposal.

Latest E2E result:

- `behave`: 13 scenarios passed, 108 steps passed.

## 8. Additional Django Tests

In addition to the browser-level E2E suite, Django tests verify important security and integration behavior.

Files:

- `core/tests/test_security.py`
- `core/tests/test_integrations.py`

Covered cases include:

- Login requirements for protected pages.
- Vote API authentication.
- Invitation access restrictions.
- Rejection of plan updates and deletions by non-creators.
- Rejection of plan creation in another user's proposal.
- Rejection of plan creation, update, and deletion for closed proposals.
- Graceful handling of plan option order conflicts.
- Normalized location-search API responses.
- Weather forecast API responses.
- Validation of missing or invalid weather coordinates.

Latest Django test result:

- `python manage.py test`: 13 tests passed.

# 9. Rubric Mapping

| Requirement | Implementation | Evidence |
|---|---|---|
| Create model instances (3 points) | Users can create `Group`, `PlanProposal`, and `Plan` instances through public application views. | `CreateView` classes, ModelForms, E2E happy paths. |
| Modify model instances (3 points) | Users can edit their own groups, proposals, and plan options. | `UpdateView` classes, ownership checks, E2E security scenarios. |
| Delete model instances (1.5 points) | Users can delete their own groups, proposals, and plan options. | `DeleteView` classes, ownership checks, E2E security scenarios. |
| External API with AJAX (2.5 points) | Plan forms use Nominatim and Open-Meteo through backend JSON endpoints. | `/api/locations/search/`, `/api/weather/forecast/`, frontend asynchronous form assistance, integration tests. |

# 10. How to Run the Project

## 10.1 Docker

```
git clone https://github.com/choosy-friends-app/choosy-app.git
cd choosy-app
docker-compose up --build
```

Then open:

```
http://localhost:8000
```

## 10.2 Local Development

```
python manage.py migrate
python manage.py seed_demo_users
python manage.py runserver
```

Then open:

```
http://localhost:8000
```

# 11. How to Run the Tests

## 11.1 Django Tests

```
python manage.py test
```

Expected result:

```
13 tests passed
```

## 11.2 End-to-End Tests

```
behave
```

Expected result:

```
13 scenarios passed, 108 steps passed
```

## 12. Evaluation Notes

The application can be evaluated entirely from the public user interface. Django admin is not required for creation, update, or deletion of the entities selected for Deliverable 2.

The main design decision was to make CRUD permissions match the real collaborative behavior of Choosy: users can manage what they create, while shared group data is protected from unauthorized modifications. The external API integration was placed in the `Plan` form because weather and location are directly relevant to deciding a group activity.

The delivery therefore extends the first version of Choosy into a Web 2.0 application with user-generated data, secure entity management, asynchronous external information, and automated E2E validation.